

Практическая работа №3

Задание:

1. Класс Phone

```
2. class Phone {
3.     String number;
4.     String model;
5.     double weight;
6.
7.     // 1. Главный конструктор с тремя параметрами – он просто принимает все
    // три поля напрямую
8.     Phone(this.number, this.model, this.weight);
9.
10.    // 2. Конструктор с двумя параметрами вызывает конструктор с тремя,
11.    // подставляя для веса значение 0.0 по умолчанию
12.    Phone.twoParams(String number, String model) : this(number, model, 0.0);
13.
14.    // 3. Конструктор без параметров вызывает конструктор с тремя,
    // подставляя дефолты
15.    Phone.empty() : this('Неизвестно', 'Неизвестно', 0.0);
16.
17.    // Метод принимает имя звонящего
18.    void receiveCall(String name) {
19.        print('Звонит $name');
20.    }
21.
22.    // Метод возвращает номер телефона
23.    String getNumber() {
24.        return number;
25.    }
26.
27.    // Метод принимает аргументы переменной длины
28.    void sendMessage(List<String> numbers) {
29.        print('Сообщение будет отправлено на следующие номера:');
30.        for (var num in numbers) {
31.            print(num);
32.        }
33.    }
34.}
35.
36.void main() {
37.    Phone phone1 = Phone('+7-999-111-22-33', 'iPhone 15', 187.0);
38.    Phone phone2 = Phone.twoParams('+7-999-444-55-66', 'Samsung S24');
39.    Phone phone3 = Phone.empty();
40.
41.    print('--- Телефон 1 ---');
42.    print('Номер: ${phone1.number}, Модель: ${phone1.model}, Вес:
    ${phone1.weight}г');
43.
44.    print('\n--- Телефон 2 ---');
```

```
45. print('Номер: ${phone2.number}, Модель: ${phone2.model}, Вес:
    ${phone2.weight}г');
46.
47. print('\n--- Телефон 3 ---');
48. print('Номер: ${phone3.number}, Модель: ${phone3.model}, Вес:
    ${phone3.weight}г');
49.
50. print('\n--- Тестирование методов ---');
51. phone1.receiveCall('Анна');
52. print('Получен номер: ${phone1.getNumber()}');
53. print('---');
54.
55. phone1.sendMessage(['+7-900-000-00-01', '+7-900-000-00-02']);
56.}
57.
```

task3.1.dart > Phone

```
1 class Phone {
2   String number;
3   String model;
4   double weight;
5
6   // 1. Главный конструктор с тремя параметрами – он просто принимает все три поля напрямую
7   Phone(this.number, this.model, this.weight);
8
9   // 2. Конструктор с двумя параметрами вызывает конструктор с тремя,
10  // подставляя для веса значение 0.0 по умолчанию
11  Phone.twoParams(String number, String model) : this(number, model, 0.0);
12
13  // 3. Конструктор без параметров вызывает конструктор с тремя, подставляя дефолты
14  Phone.empty() : this('Неизвестно', 'Неизвестно', 0.0);
15
16  // Метод принимает имя звонящего
17  void receiveCall(String name) {
18    print('Звонит $name');
19  }
20
21  // Метод возвращает номер телефона
22  String getNumber() {
23    return number;
24  }
25
26  // Метод принимает аргументы переменной длины
27  void sendMessage(List<String> numbers) {
28    print('Сообщение будет отправлено на следующие номера:');
29    for (var num in numbers) {
30      print(num);
31    }
32  }
33 }
34
35 void main() {
36   Phone phone1 = Phone('+7-999-111-22-33', 'iPhone 15', 187.0);
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ POSTMAN CONSOLE

+ v powershell

PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> dart task3.1.dart

--- Телефон 1 ---

Номер: +7-999-111-22-33, Модель: iPhone 15, Вес: 187.0г

--- Телефон 2 ---

Номер: +7-999-444-55-66, Модель: Samsung S24, Вес: 0.0г

--- Телефон 3 ---

Номер: Неизвестно, Модель: Неизвестно, Вес: 0.0г

--- Тестирование методов ---

Звонит Анна

Получен номер: +7-999-111-22-33

Сообщение будет отправлено на следующие номера:

+7-900-000-00-01

+7-900-000-00-02

PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2>

2. Класс Матрица

```
class Matrix {
    List<List<double>> data;
    int rows;
    int columns;

    // Конструктор класса
    Matrix(this.data)
        : rows = data.length,
          columns = data.isNotEmpty ? data[0].length : 0;

    // Метод вывода на печать
    void printMatrix() {
        for (var row in data) {
            print(row.map((val) => val.toStringAsFixed(1)).join('\t'));
        }
    }

    // Метод сложения с другой матрицей
    Matrix add(Matrix other) {
        if (rows != other.rows || columns != other.columns) {
            throw ArgumentError('Размеры матриц должны совпадать для сложения.');
        }

        List<List<double>> resultData = List.generate(rows, (i) =>
            List.generate(columns, (j) => data[i][j] + other.data[i][j])
        );

        return Matrix(resultData);
    }

    // Метод умножения на число
    Matrix multiplyByScalar(double scalar) {
        List<List<double>> resultData = data.map((row) =>
            row.map((val) => val * scalar).toList()
        ).toList();

        return Matrix(resultData);
    }

    // Метод умножения матриц
    Matrix multiply(Matrix other) {
        if (columns != other.rows) {
            throw ArgumentError('Количество столбцов первой матрицы должно быть равно количеству строк второй матрицы.');
        }

        List<List<double>> resultData = List.generate(rows, (_) =>
            List.filled(other.columns, 0.0));
    }
}
```

```

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < other.columns; j++) {
                double sum = 0;
                for (int k = 0; k < columns; k++) {
                    sum += data[i][k] * other.data[k][j];
                }
                resultData[i][j] = sum;
            }
        }

        return Matrix(resultData);
    }
}

void main() {
    // Создаем тестовые матрицы
    Matrix matrixA = Matrix([
        [1.0, 2.0],
        [3.0, 4.0]
    ]);

    Matrix matrixB = Matrix([
        [5.0, 6.0],
        [7.0, 8.0]
    ]);

    print('--- Матрица A (${matrixA.rows}x${matrixA.columns}) ---');
    matrixA.printMatrix();

    print('\n--- Матрица Б (${matrixB.rows}x${matrixB.columns}) ---');
    matrixB.printMatrix();

    print('\n1. Результат сложения (A + Б):');
    Matrix sumResult = matrixA.add(matrixB);
    sumResult.printMatrix();

    print('\n2. Результат умножения матрицы A на число 3:');
    Matrix scalarResult = matrixA.multiplyByScalar(3);
    scalarResult.printMatrix();

    print('\n3. Результат умножения матриц (A * Б):');
    Matrix multiplyResult = matrixA.multiply(matrixB);
    multiplyResult.printMatrix();
}

```

```
task3.2.dart > main
1  class Matrix {
60 }
61
Run | Debug
62 void main() {
63     // Создаем тестовые матрицы
64     Matrix matrixA = Matrix([
65         [1.0, 2.0],
66         [3.0, 4.0]
67     ]);
68
69     Matrix matrixB = Matrix([
70         [5.0, 6.0],
71         [7.0, 8.0]
72     ]);
73
74     print('--- Матрица A (${matrixA.rows}x${matrixA.columns}) ---');
75     matrixA.printMatrix();
76
77     print('\n--- Матрица B (${matrixB.rows}x${matrixB.columns}) ---');
78     matrixB.printMatrix();
79
80     print('\n1. Результат сложения (A + B):');
81     Matrix sumResult = matrixA.add(matrixB);
82     sumResult.printMatrix();
83
84     print('\n2. Результат умножения матрицы A на число 3:');
85     Matrix scalarResult = matrixA.multiplyByScalar(3);
86     scalarResult.printMatrix();
87
88     print('\n3. Результат умножения матриц (A * B):');
89     Matrix multiplyResult = matrixA.multiply(matrixB);
90     multiplyResult.printMatrix();
91 }
92
```

ПРОБЛЕМЫ Выходные данные КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ POSTMAN CONSOLE + v powershell

```
PS D:\Учеба\3 курс\ВДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> dart task3.2.dart
--- Матрица A (2x2) ---
1.0  2.0
3.0  4.0

--- Матрица B (2x2) ---
5.0  6.0
7.0  8.0

1. Результат сложения (A + B):
6.0  8.0
10.0 12.0

2. Результат умножения матрицы A на число 3:
3.0  6.0
9.0  12.0

3. Результат умножения матриц (A * B):
19.0  22.0
43.0  50.0
PS D:\Учеба\3 курс\ВДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2>
```

3. Рекурсивный вывод чисел

```
import 'dart:io';

// Рекурсивная функция вывода чисел
void printNumbersRecursive(int a, int b) {
  print(a);

  if (a == b) {
    return;
  }

  if (a < b) {
    printNumbersRecursive(a + 1, b);
  } else {
    printNumbersRecursive(a - 1, b);
  }
}

void main() {
  print('--- Рекурсивный вывод чисел ---');

  // Ввод числа A
  stdout.write('Введите число A: ');
  String inputA = stdin.readLineSync() ?? '';
  int? a = int.tryParse(inputA);

  // Ввод число B
  stdout.write('Введите число B: ');
  String inputB = stdin.readLineSync() ?? '';
  int? b = int.tryParse(inputB);

  // Проверка на корректность ввода
  if (a == null || b == null) {
    print('Ошибка: вводимые значения должны быть целыми числами!');
    return;
  }

  print('\nРезультат вывода:');
  printNumbersRecursive(a, b);
}
```

task3.3.dart

```
1 import 'dart:io';
2
3 // Рекурсивная функция вывода чисел
4 void printNumbersRecursive(int a, int b) {
5     print(a);
6
7     if (a == b) {
8         return;
9     }
10
11     if (a < b) {
12         printNumbersRecursive(a + 1, b);
13     } else {
14         printNumbersRecursive(a - 1, b);
15     }
16 }
17
18 Run | Debug
19 void main() {
20     print('--- Рекурсивный вывод чисел ---');
21
22     // Ввод числа A
23     stdout.write('Введите число A: ');
24     String inputA = stdin.readLineSync() ?? '';
25     int? a = int.tryParse(inputA);
26
27     // Ввод число B
28     stdout.write('Введите число B: ');
29     String inputB = stdin.readLineSync() ?? '';
30     int? b = int.tryParse(inputB);
31
32     // Проверка на корректность ввода
33     if (a == null || b == null) {
34         print('Ошибка: вводимые значения должны быть целыми числами!');
35     }
36 }
```

task3.1.darttask3.2.darttask3.3.darttask3.4.dart

ПРОБЛЕМЫ

ВЫХОДНЫЕ ДАННЫЕ

КОНСОЛЬ ОТЛАДКИ

ТЕРМИНАЛ

ПОРТЫ

POSTMAN CONSOLE

+ powershell

🗑

...

🔄

✕

● PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> dart task3.3.dart

--- Рекурсивный вывод чисел ---

Введите число A: 10

Введите число B: 20

Результат вывода:

10

11

12

13

14

15

16

17

18

19

20

● PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> dart task3.3.dart

--- Рекурсивный вывод чисел ---

Введите число A: 20

Введите число B: 10

Результат вывода:

20

19

18

17

16

15

14

13

12

11

10

● PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2>

4. Наследование Student, Aspirant

```
// Базовый класс Студент
class Student {
    String firstName;
    String lastName;
    String group;
    double averageMark;

    // Конструктор класса Student
    Student(this.firstName, this.lastName, this.group, this.averageMark);

    // Метод для расчета стипендии студента
    int getScholarship() {
        return averageMark == 5 ? 2000 : 1900;
    }
}

// Класс Аспирант, который наследуется от Студента
class Aspirant extends Student {
    String scientificWork; // Научная работа

    // Конструктор класса Aspirant, вызывающий конструктор базового класса через
    super
    Aspirant(
        String firstName,
        String lastName,
        String group,
        double averageMark,
        this.scientificWork
    ) : super(firstName, lastName, group, averageMark);

    // Переопределение метода расчета стипендии для аспиранта
    @override
    int getScholarship() {
        return averageMark == 5 ? 2500 : 2200;
    }
}

void main() {
    // Создание массива (List) типа Student, содержащего и студентов, и аспирантов
    List<Student> students = [
        Student('Иван', 'Иванов', 'ИСП-401', 4.8),
        Student('Мария', 'Петрова', 'ИСП-402', 5.0),
        Aspirant('Алексей', 'Сидоров', 'А-101', 4.5, 'Исследование ИИ во Flutter'),
        Aspirant('Елена', 'Козлова', 'А-102', 5.0, 'Оптимизация баз данных NoSQL'),
    ];

    print('--- Расчет стипендий ---');

    // Вызов метода getScholarship() для каждого элемента массива
```

```

for (var person in students) {
    String role = person is Aspirant ? 'Аспирант' : 'Студент';

    print('$role: ${person.firstName} ${person.lastName}');
    print('Средняя оценка: ${person.averageMark}');
    print('Стипендия: ${person.getScholarship()} руб.');
```

```

    print('-----');
}
}

```

```

task3.4.dart > ...
1 // Базовый класс Студент
2 class Student {
3     String firstName;
4     String lastName;
5     String group;
6     double averageMark;
7
8     // Конструктор класса Student
9     Student(this.firstName, this.lastName, this.group, this.averageMark);
10
11     // Метод для расчета стипендии студента
12     int getScholarship() {
13         return averageMark == 5 ? 2000 : 1900;
14     }
15 }
16
17 // Класс Аспирант, который наследуется от Студента
18 class Aspirant extends Student {
19     String scientificWork; // Научная работа
20
21     // Конструктор класса Aspirant, вызывающий конструктор базового класса через super
22     Aspirant(
23         String firstName,
24         String lastName,
25         String group,
26         double averageMark,
27         this.scientificWork
28     ) : super(firstName, lastName, group, averageMark);
29
30     // Переопределение метода расчета стипендии для аспиранта
31     @override
32     int getScholarship() {
33         return averageMark == 5 ? 2500 : 2200;
34     }

```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ POSTMAN CONSOLE

```

PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> dart task3.4.dart
--- Расчет стипендий ---
Студент: Иван Иванов
Средняя оценка: 4.8
Стипендия: 1900 руб.
-----
Студент: Мария Петрова
Средняя оценка: 5.0
Стипендия: 2000 руб.
-----
Аспирант: Алексей Сидоров
Средняя оценка: 4.5
Стипендия: 2200 руб.
-----
Аспирант: Елена Козлова
Средняя оценка: 5.0
Стипендия: 2500 руб.
-----
PS D:\Учеба\3 курс\МДК 01.03 Разработка мобильных приложений\Практические работы\Практическая работа 2> 

```